

Most Profit Assigning Work

There are n jobs available. Each job is characterized by a difficulty and a profit. Specifically, the i -th job has difficulty `difficultyi` and yields a profit of `profiti`.

There are also m workers, where the j -th worker has an ability of `workerj`. A worker can successfully complete any job whose difficulty is less than or equal to their ability.

Each worker can be assigned at most one job. However, a single job can be assigned to multiple workers. If a worker is not assigned a job, they do not earn any profit.

You are given the integers n and m , and the arrays `difficulty`, `profit`, and `worker`. Compute the maximum total profit that can be earned by assigning jobs to workers.

Example Test Cases:

Test Case 1

Input: $n = 5$, `difficulty` = [2,4,6,8,10], `profit` = [10,20,30,40,50]
 $m = 4$, `worker` = [4,5,6,7]

Output: 100

Explanation: The first two workers are assigned the job of difficulty 4, and the last two workers are assigned the job of difficulty 6. The total profit is $2(20) + 2(30) = 100$.

Test Case 2

Input: $n = 3$, `difficulty` = [85,47,57], `profit` = [24,66,99]
 $m = 3$, `worker` = [40,25,25]

Output: 0

Explanation: The easiest job requires difficulty 47. All workers have abilities below 47, so no jobs can be completed.

Expected Time Complexity: $O(n \log n + m \log n)$

Input Format:

An integer n , and two arrays `difficulty` and `profit` of length n ; additionally, an integer m , and an array `worker` of length m .

Output Format:

Return a single integer, the maximum total profit.

Constraints:

1. $1 \leq n, m \leq 10^4$
2. $1 \leq \text{difficulty}_i, \text{profit}_i, \text{worker}_i \leq 10^5$

Instructions:

You have been provided with two template files, containing the `main` and `solve` functions respectively. The `main` function, which has been completed for you, parses the input for each test case. For each test case, it calls the `solve` function, which must compute the maximum total profit.

The `solve` function must compute this in $O(n \log n + m \log n)$ time.

(Refer to the `Instructions.pdf` file for detailed guidelines on the input-output format and additional test cases.)

Library Usage

For this lab, you are permitted to use any modules which are part of the standard library of either `C++` or `python`. For `C++`, do not add any extra `#include` directives; the provided `bits/stdc++.h` covers all allowed modules.